Projet - NoSQL Databases (LOCAL)

JO Milano–Cortina 2026

Exploration et requêtes NoSQL avec Python
MongoDB + Neo4j Desktop

Année : 2025–2026 | Durée : 21h

Travail : En Groupe de 5 Max (selon consigne du Professeur)
Projet propose par : Mr TOUATI Zouhaier

0) Objectif du projet

Développer une application Python qui se connecte en local à MongoDB et Neo4j Desktop, exécute des requêtes, répond à des questions imposées, et génère des visualisations.

1) Compétences visées

- Connexion locale sécurisée et configuration propre (variables d'environnement / fichier .env).
- MongoDB : requêtes, agrégations et opérations CRUD (insert / update / delete).
- Neo4j : modélisation graphe et requêtes Cypher.
- Analyse et visualisation des résultats (Python + visualisation graphe).
- Gestion de projet en Agile/Scrum (preuves dans le rapport).

2) Prérequis et outils (LOCAL)

Pourquoi : uniformiser l'environnement et éviter les blocages techniques.

À faire : installer et lancer les services locaux suivants :

- MongoDB local : URI = mongodb://localhost:27017
- Neo4j Desktop : démarrer le DBMS dans Neo4j Desktop (URI souvent : bolt://localhost:7687)
- Python 3.10+ recommandé

Bibliothèques conseillées :

- pymongo (MongoDB)
- neo4j (driver officiel)
- matplotlib (ou seaborn) pour les graphiques
- pyvis (ou Neovis.js) pour visualiser le graphe
- Option UI (bonus) : streamlit

3) Contexte métier (JO 2026)

Une plateforme permet à des utilisateurs (staff, bénévoles, journalistes, fans) de publier des tweets liés à l'organisation des JO : transport, sécurité, météo, sites, annonces, incidents, etc. Les tweets peuvent contenir des hashtags, recevoir des favoris (likes), être des réponses, et être retweetés. Des relations de suivi (follows) existent entre utilisateurs.

4) Données à créer (modèle imposé)

4.1 MongoDB - Base milano2026

Créer une base de données MongoDB nommée milano2026 avec 2 collections obligatoires :

Collection users

Champs recommandés :

- user_id (unique)
- username
- role (ex : staff, volunteer, journalist, fan)
- country
- created_at

Collection tweets

Champs recommandés :

- tweet_id (unique)
- user_id (auteur)
- text
- hashtags (liste) ex : ["milano2026", "transport"]
- created_at
- favorite_count (likes)
- in_reply_to_tweet_id (null si tweet initial)

CRUD obligatoire (à implémenter en Python) :

- insert_user(), update_user(), delete_user()
- insert_tweet(), update_tweet(), delete_tweet()

4.2 Neo4j Desktop - Modèle graphe

Créer dans Neo4j les nœuds et relations suivants (labels imposés) :

Nœuds

- (:User {user_id, username})
- (:Tweet {tweet_id})

Relations (minimum requis)

- (:User)-[:FOLLOWS]->(:User)
- (:User)-[:RETWEETS]->(:Tweet)
- (:User)-[:AUTHORED]->(:Tweet) (recommandé)
- (:Tweet)-[:REPLY_TO]->(:Tweet) (threads / discussions)

Principe : MongoDB contient les données complètes (texte, hashtags, likes, dates) et Neo4j contient principalement les relations en s'appuyant sur les IDs (user_id, tweet_id).

5) Dataset (à fournir obligatoirement)

Fournir un dataset reproductible, au choix :

- Option A : users.json + tweets.json + script import_data.py
- Option B (recommandée) : script seed.py qui génère et insère les données

Contraintes minimales du dataset :

- Au moins 30 utilisateurs et 200 tweets
- Hashtags variés (dont milano2026)
- Des réponses (in_reply_to_tweet_id) pour créer des conversations
- Des liens FOLLOWS et RETWEETS dans Neo4j
- Un utilisateur spécial : MilanoOps

6) Travail demandé

Votre application Python doit :

1. Se connecter aux deux bases locales (MongoDB + Neo4j Desktop).
2. Charger ou générer le dataset (import ou seed).
3. Exécuter les requêtes demandées et afficher les résultats (console ou UI).
4. Produire des visualisations (MongoDB + graphe Neo4j).

7) Questions à traiter (obligatoires)

1. Donner le nombre d'utilisateurs.
2. Donner le nombre de tweets.
3. Donner le nombre de hashtags distincts.
4. Donner le nombre de tweets contenant un hashtag donné (hashtag en paramètre).
5. Donner le nombre d'utilisateurs distincts ayant tweeté un tweet contenant le hashtag milano2026.
6. Donner les tweets qui sont des réponses à un autre tweet.
7. Donner le nom des followers de MilanoOps (dans ce dataset).
8. Donner le nom des utilisateurs suivis par MilanoOps (dans ce dataset).
9. Donner le nom des utilisateurs en relation réciproque avec MilanoOps.
10. Donner les utilisateurs avec plus de 10 followers.
11. Donner les utilisateurs qui suivent plus de 5 utilisateurs.
12. Donner les 10 tweets les plus populaires (popularité = favorite_count).
13. Donner les 10 hashtags les plus populaires (popularité = nombre de tweets où ils apparaissent).
14. Donner les tweets qui initient une discussion (début d'un thread REPLY_TO).
15. Quelle est la discussion la plus longue ?
16. Pour chaque conversation, donner le début et la fin (tweet de début + tweet de fin).

8) Analyse et visualisation (obligatoire)

8.1 Visualisations MongoDB (Python)

- KPI : nombre d'utilisateurs / nombre de tweets / nombre de hashtags
- Top 10 hashtags (bar chart)
- Top 10 tweets (bar chart)

8.2 Visualisation Neo4j (graphe)

- Afficher le réseau autour de MilanoOps (followers / following) avec pyvis (ou Neovis.js).

9) Agilité (Scrum) sur 21h

Organiser le travail en sprints et fournir des preuves Scrum dans le rapport.

Planning conseillé (21h)

- Sprint 0 - 3h : cadrage, backlog, DoD/DoR, tests de connexion, stratégie dataset.
- Sprint 1 - 7h : MongoDB (modèle, CRUD, questions 1 à 6, 12 à 13, graphiques).
- Sprint 2 - 7h : Neo4j (modèle graphe, import IDs, questions 7 à 11, 14 à 16, visualisation).
- Sprint 3 - 4h : finalisation (tests, README, rapport, captures, démo).

Artefacts Scrum obligatoires dans le rapport

- Product backlog (user stories priorisées).
- Sprint backlogs (tâches par sprint).
- Mini daily log (fait / à faire / blocages).
- Sprint review + rétro (1 amélioration actionnable).

10) Livrables attendus

1. Rapport PDF (voir structure conseillée).
2. Code source Python commenté + requirements.txt.
3. Données + script seed.py ou import_data.py.
4. README : installation locale, lancement, exemples d'exécution, captures.

Structure conseillée du rapport

- Introduction et objectifs
- Environnement local (MongoDB, Neo4j Desktop) + captures de connexions
- Modèle MongoDB (users/tweets)
- Modèle Neo4j (labels/relations) + schéma
- Implémentation Python (architecture, modules)
- Requêtes (MongoDB + Cypher) : explications + résultats
- Visualisations : graphiques + interprétation
- Agile/Scrum : backlog, sprints, daily log, review, rétro
- Difficultés rencontrées + solutions
- Conclusion

11) Barème proposé (/20)

Critère	Points
Connexions locales + configuration propre (.env / README)	3
MongoDB : CRUD + requêtes + agrégations correctes	6
Neo4j : modèle + Cypher + questions réseau/discussions	6
Visualisations (MongoDB + graphe)	2
Agile/Scrum (preuves : backlog, sprints, logs, rétro)	2
Qualité globale (structure code, commentaires, clarté rapport)	1

12) Bonus (optionnel)

- Mini dashboard Streamlit : filtres hashtag, top tweets, threads.
- Export CSV des réponses.
- Visualisation avancée des conversations (threads) en graphe.
- Seed paramétrable (nb users, nb tweets, taux replies/retweets).

Annexe - Templates rapides

Definition of Done (DoD) - exemple

- Code exécuté sans erreur
- Résultat correct et affiché
- Fonction documentée (docstring/commentaire)
- Preuve dans le rapport (capture ou sortie console)

Daily log - format

- Fait : ...
- À faire : ...
- Blocage : ...

User story - exemple

En tant que responsable ops, je veux afficher le top 10 hashtags, afin d'identifier les sujets dominants.

Critères d'acceptation - exemple

- Le top 10 est trié par nombre d'apparitions (décroissant).
- L'affichage contient le hashtag et sa fréquence.
- Une capture d'écran ou sortie console est incluse dans le rapport.

Sprint backlog - format simple

- Tâche - Estimation - Statut (To do / Doing / Done)
- Lien vers la user story associée (si vous utilisez un outil type Trello/Jira)